

Remote Testing & Staging Deployment Guide

Setup on Remote Machine (PLAN B)

Prerequisite: Remote machine has:

- ✓ Service account credentials (sa-tot-osa-key.json)
- ✓ Python 3.10+
- ✓ gcloud CLI configured
- ✓ Git access

Step 1: Clone & Setup

```
cd /path/to/remote/workspace
git clone <repo-url>
cd intelligent-shelf-agents

# Setup Python environment
python3 -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\Activate.ps1

# Install dependencies
pip install -r requirements.txt
pip install -r ispirot-api/requirements.txt
```

Step 2: Set GCP Credentials

```
# Get the path to your SA key file
export GOOGLE_APPLICATION_CREDENTIALS="/path/to/sa-tot-osa-key.json"

# Verify gcloud is configured
gcloud auth activate-service-account sa-tot-osa@corp-stro-salesinventory-
prod.iam.gserviceaccount.com \
  --key-file=$GOOGLE_APPLICATION_CREDENTIALS

gcloud config set project corp-stro-salesinventory-prod

# Verify access
gcloud ai reasoning-engines list --project=corp-stro-salesinventory-prod --
region=us-central1
```

Expected Output:

NAME	ID
ispilot-reasoning-engine	5375474415045705728

Step 3: Run Integration Tests

```
cd intelligent-shelf-agents

# This will test:
# 1. GCP auth setup
# 2. Module imports (agents, services)
# 3. Agent configuration
# 4. Live BigQuery queries
python test_local_integration.py
```

Expected Output (if successful):

```
=====
ISpilot Integration Test Suite
=====

=== Test 0: GCP Authentication ===
✓ GOOGLE_APPLICATION_CREDENTIALS set: /path/to/sa-tot-osa-key.json
✓ gcloud project: corp-stro-salesinventory-prod
  ✓ Correct project!

=== Test 1: Module Imports ===
✓ Coordinator agent loaded: ispilot_coordinator
✓ AnalyticsService imported
✓ StoreService imported

=== Test 2: Analytics Service ===
Testing Chile (CL)...
✓ Analytics service call succeeded (metrics logged)

=== Test 3: Agent Configuration ===
Agent name: ispilot_coordinator
Subagents: ['ispilot_analyst', 'ispilot_coach']
✓ Agent configuration valid

=====
✓ Integration test complete!
=====
```

Metrics Output Should Show:

```
✓ [ANALYTICS] get_latest_daily_summary(CL) took 1234.56ms
✓ [ANALYTICS] get_latest_daily_summary(PE) took 890.12ms
```

Step 4: Verify Metrics Are Working

Check that:

1. ☒ Console output shows latency logs (✓ [SERVICE] operation took X.XXms)
2. ☒ No exceptions thrown
3. ☒ BigQuery queries successful
4. ☒ Agent configs load correctly

If test passes: Proceed to Step 5 (Staging Deployment)

If test fails: Debug and document error in BLOCKERS.md

Deploy to Staging

Step 5: Build & Push Docker Image

From **ispilot-api** directory:

```
cd ispilot-api

# Build for staging
docker build -t gcr.io/corp-stro-salesinventory-prod/ispilot-api:staging .

# Push to Container Registry
docker push gcr.io/corp-stro-salesinventory-prod/ispilot-api:staging
```

Or use Cloud Build (faster, no local Docker):

```
gcloud builds submit \
  --tag=gcr.io/corp-stro-salesinventory-prod/ispilot-api:staging \
  --project=corp-stro-salesinventory-prod
```

Step 6: Deploy to Cloud Run Staging

```
gcloud run deploy ispilot-api-staging \
  --image=gcr.io/corp-stro-salesinventory-prod/ispilot-api:staging \
  --project=corp-stro-salesinventory-prod \
  --region=us-central1 \
  --service-account=sa-tot-osa@corp-stro-salesinventory-
prod.iam.gserviceaccount.com \
```

```
--no-allow-unauthenticated \
--set-env-vars="GOOGLE_CLOUD_PROJECT=corp-stro-salesinventory-
prod,VERTEX_ENGINE_ID=5375474415045705728,ENVIRONMENT=staging,LOG_LEVEL=DEBUG,EN
ABLE_METRICS=true" \
--memory=1Gi \
--cpu=1 \
--timeout=120 \
--wait-for-deployment
```

Expected Output:

```
Service [ispilot-api-staging] revision [ispilot-api-staging-00001-xxx] has been
deployed
https://ispilot-api-staging-xxxxx.run.app
```

Step 7: Test Staging Endpoint

```
# Get the staging URL
STAGING_URL=$(gcloud run services describe ispilot-api-staging \
--project=corp-stro-salesinventory-prod \
--region=us-central1 \
--format='value(status.url)')

# Test health check
curl -X GET "$STAGING_URL/health"

# Expected:
# {"status": "healthy", ...}
```

Step 8: Test Chat Endpoint (with Auth)

You'll need an API key or OAuth token. For staging:

```
# Using a test API key (ask team lead)
API_KEY="your-staging-api-key"

curl -X POST "$STAGING_URL/chat" \
-H "Content-Type: application/json" \
-H "X-API-Key: $API_KEY" \
-d '{
  "user_id": "test-user-sprint3",
  "message": "How is Talca Colin performing?",
  "session_id": "test-session-sprint3"
}' | jq .

# Expected response:
```

```
# {
#   "status": "success",
#   "session_id": "test-session-sprint3",
#   "response": "The Talca Colin store shows...",
#   "metadata": {
#     "latency_ms": 5234,
#     "agents_used": ["shelf_analyst"],
#     "timestamp": "2026-08-29T..."
#   }
# }
```

Step 9: Load Testing

```
# Install Apache Bench (if needed)
# Mac: brew install httpd
# Linux: sudo apt-get install apache2-utils
# Windows: choco install apache-bench

# Create payload file
cat > payload.json << 'EOF'
{
  "user_id": "load-test-user",
  "message": "How is Talca Colin performing?",
  "session_id": "load-test-session"
}
EOF

# Run 100 requests with 10 concurrent
ab -n 100 -c 10 \
  -H "X-API-Key: $API_KEY" \
  -p payload.json \
  "$STAGING_URL/chat"

# Collect metrics:
# - Requests per second
# - Mean latency
# - Percentiles (p95, p99)
# - Error rate
```

Step 10: Check Logs

```
# View Cloud Run logs for staging service
gcloud logging read \
  'resource.type="cloud_run_revision" AND
resource.labels.service_name="ispilot-api-staging"' \
  --project=corp-stro-salesinventory-prod \
  --limit=50 \
  --format=json | jq .
```

```
# Filter for metrics
gcloud logging read \
  'resource.type="cloud_run_revision" AND textPayload=~"METRICS"' \
  --project=corp-stro-salesinventory-prod \
  --limit=20 \
  --format=json | jq '.[] | .textPayload'
```

Expected Metrics in Logs:

```
[AGENT_METRICS] ispiilot_coordinator.route latency_ms=1234.56
[ANALYTICS_METRICS] get_latest_daily_summary(CL) latency_ms=2345.67
[VERTEX_METRICS] agent_invoke latency_ms=3456.78
```

Success Criteria - Check Before Moving to Production

Staging Validation Checklist ☒

- ☐ **Integration Test** (`test_local_integration.py` runs on remote, no errors)
- ☐ **Metrics Output** (Console shows latency logs: ✓ `[SERVICE] operation took X.XXms`)
- ☐ **Docker Build** (Image builds successfully, pushed to registry)
- ☐ **Cloud Run Deploy** (Service deployed, health check OK)
- ☐ **API Response** (Chat endpoint returns valid JSON with metrics)
- ☐ **Load Test** (10 concurrent users, < 5% error rate)
- ☐ **Latency Budget** (p99 latency < 2000ms for agent operations)
- ☐ **Logging** (Cloud Logging shows structured metrics)
- ☐ **Error Handling** (Test with invalid input, see proper error codes)
- ☐ **Fallback Paths** (Test with BigQuery timeout simulated)

If all checkmarks ☒:

- Proceed to production deployment
- Document baseline metrics
- Update SLA targets

If any fails ✗:

- Document in BLOCKERS.md
- Identify root cause
- Fix and re-test
- Create follow-up issue

Troubleshooting Remote Testing

Problem: "No module named 'google.adk'"

Solution: Install root dependencies

```
cd intelligent-shelf-agents
pip install -r requirements.txt
```

Problem: "Authentication failed"

Solution: Verify SA setup

```
gcloud auth list
gcloud config list
gcloud auth application-default print-access-token
```

Problem: "Failed to connect to BigQuery"

Solution: Verify permissions

```
bq ls --project_id=corp-stro-salesinventory-prod
bq show corp-stro-salesinventory-prod:is_data
```

Problem: "VertexAgentClient invocation failed"

Solution: Check reasoning engine

```
gcloud ai reasoning-engines list \
  --project=corp-stro-salesinventory-prod \
  --region=us-central1
```

Files to Review on Remote Machine

After cloning, these are key files to understand:

1. Metrics Integration Points:

- `agents/coordinator/agent.py` (lines 1-35)
- `services/analytics_service.py` (lines 1-10, 32-105)
- `services/store_service.py` (lines 1-10, 23-110)
- `ispilot-api/app/services/vertex_client.py` (lines 150-190)

2. Test Script:

- `test_local_integration.py` (entire file)

3. Documentation:

- [WORK_CHECKPOINT.md](#) (Sprint 3 section)
- [SPRINT3_DETAILED_PLAN.md](#) (full context)

Next: Production Deployment (After Staging Validation)

Once staging tests pass:

1. Tag staging image as **latest**
2. Deploy to Cloud Run production
3. Run smoke tests against prod
4. Monitor metrics for 24 hours
5. Declare success and close sprint

```
# After staging validation passes:

# Tag as latest
docker tag \
  gcr.io/corp-stro-salesinventory-prod/ispilot-api:staging \
  gcr.io/corp-stro-salesinventory-prod/ispilot-api:latest

docker push gcr.io/corp-stro-salesinventory-prod/ispilot-api:latest

# Deploy to production (same as staging, different service name)
gcloud run deploy ispilot-api \
  --image=gcr.io/corp-stro-salesinventory-prod/ispilot-api:latest \
  --project=corp-stro-salesinventory-prod \
  --region=us-central1 \
  --service-account=sa-tot-osa@corp-stro-salesinventory-
prod.iam.gserviceaccount.com \
  --no-allow-unauthenticated \
  --set-env-vars="GOOGLE_CLOUD_PROJECT=corp-stro-salesinventory-
prod,VERTEX_ENGINE_ID=5375474415045705728,ENVIRONMENT=production,LOG_LEVEL=INFO
,ENABLE_METRICS=true" \
  --memory=1Gi \
  --cpu=1 \
  --timeout=120 \
  --wait-for-deployment
```

Status: Ready for Remote Testing

Last Updated: 2026-08-29

Contact: [Your Name] for questions